E-Commerce Analytics Platform

Technical Project Report

Course: Advanced Application Development (CSE 214)

Multi-Agent Text2SQL AI Chatbot & Multi-Role E-Commerce Platform

Yaren Erişen 20240808601

Elif Polatdemir 20230808612

GitHub Repository: https://github.com/elifpdemir/ecommerce

1. Executive Summary

This project delivers a full-stack e-commerce analytics platform inspired by modern business intelligence tools such as Metabase. The system supports three distinct user roles — Admin, Corporate (Seller), and Individual — each provided with a tailored dashboard and AI-powered analytical capabilities. The platform is built on a normalized relational database assembled from six Kaggle datasets and is augmented with a multi-agent Text2SQL chatbot that translates natural language questions into executable SQL queries.

1.1 Technology Stack

Layer	Technology	Version / Notes
Backend	Spring Boot	3.2.5 (Java 21 LTS)
Frontend	Angular	SPA — TypeScript / RxJS
Database	MySQL / MariaDB	10.4.28 — PostgreSQL compatible
AI Integration	Google Gemini 2.5 Flash	Text2SQL via REST (RestTemplate)
Payments	Stripe API	Test mode; card processing
Visualization	Chart.js & Plotly	Angular-embedded analytics charts
Auth	JWT (JJWT 0.11.5)	HS256, 24-hour tokens
Build	Maven	Spring parent POM

1.2 Key Capabilities

- Multi-tenant design with Role-Based Access Control (RBAC) for Admin, Corporate, and Individual roles
- JWT-based stateless authentication with refresh token infrastructure
- Full CRUD REST API across 9 core entities via 11 Spring controllers
- Natural language query → SQL generation → secure execution → formatted Markdown response pipeline
- Stripe API integration for checkout and payment processing
- Interactive analytics dashboards powered by Chart.js and Plotly
- Automatic blocking of sensitive fields (e.g. password_hash) in AI response flows
- ETL integration of six Kaggle datasets into a single 3NF-normalized schema

2. System Architecture and Design Decisions

2.1 High-Level Architecture

The system follows a classic 3-tier architecture in which each layer can be scaled and deployed independently. The Presentation Layer is an Angular Single-Page Application; the Application Layer is a Spring Boot service exposing REST endpoints; and the Data Layer is a MySQL/MariaDB relational database accessed via JPA/Hibernate for entity operations and JdbcTemplate for the dynamic AI-generated queries.

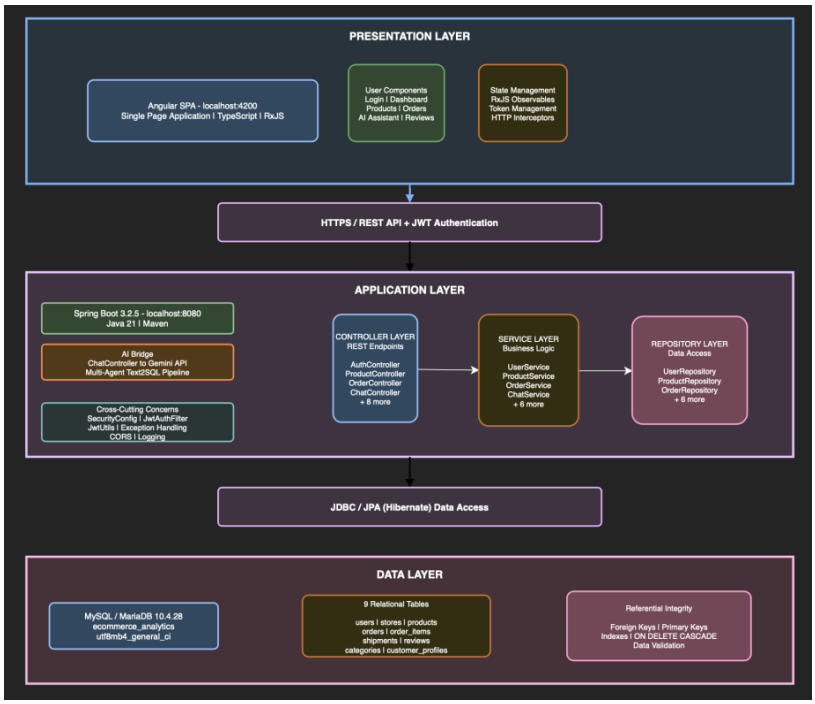

Figure 1 — Three-Tier System Architecture Diagram

2.2 Architectural Decision Matrix

Decision	Selected Solution	Rationale
Backend Framework	Spring Boot 3.2.5	Mature ecosystem; seamless Spring Security RBAC; multi-database JPA support
Java Version	Java 21 (LTS)	Virtual threads, modern language features, long-term support
Authentication	Stateless JWT (HS256)	Horizontal scaling friendly; no server-side session; ideal for SPAs
Data Access	JPA + JdbcTemplate	JPA for CRUD; JdbcTemplate for dynamic AI-generated queries
Database	MySQL / MariaDB	phpMyAdmin management; broad hosting support; portable to PostgreSQL
AI Provider	Google Gemini 2.5 Flash	Low latency, free tier, JSON generative text API
Frontend	Angular	Modular components; RxJS async data flow; enterprise-grade
Payments	Stripe API	Industry-standard checkout; test mode cards; easy live migration
Visualization	Chart.js & Plotly	Chart.js for dashboards; Plotly for rich interactive analytics
API Style	REST + JSON	Client-independent; documented with OpenAPI/Swagger

2.3 Package Structure

The Spring Boot project uses a standard layered package structure under com.example.ecommerce_project:

Layer	Package Content	Description
Config	SecurityConfig	Spring Security and CORS configuration
Security	JwtUtils, JwtAuthFilter	JWT token creation, validation, and filter chain
Entity	User, Product, Order, …	JPA entity classes mapped to database tables
Repository	Spring Data JPA interfaces	CRUD and custom query methods
Service	UserService, ChatService, …	Business logic and Text2SQL orchestration
Controller	AuthController, ChatController, …	REST endpoint handlers (11 controllers)
DTO	LoginRequest, ChatResponse, …	Data transfer objects between layers

3. Database Design and ER Diagram

3.1 Schema Overview

The database contains 9 tables normalized to Third Normal Form (3NF). All foreign keys define explicit ON DELETE behaviors, and indexes are added to frequently queried fields (e.g. idx_user_email on users.email). The storage engine is InnoDB with utf8mb4_general_ci collation to support Turkish and international character sets. While the primary deployment target is MySQL/MariaDB 10.4.28, the schema and JDBC connection string are compatible with PostgreSQL through a configuration change in application.properties.

3.2 ER Diagram

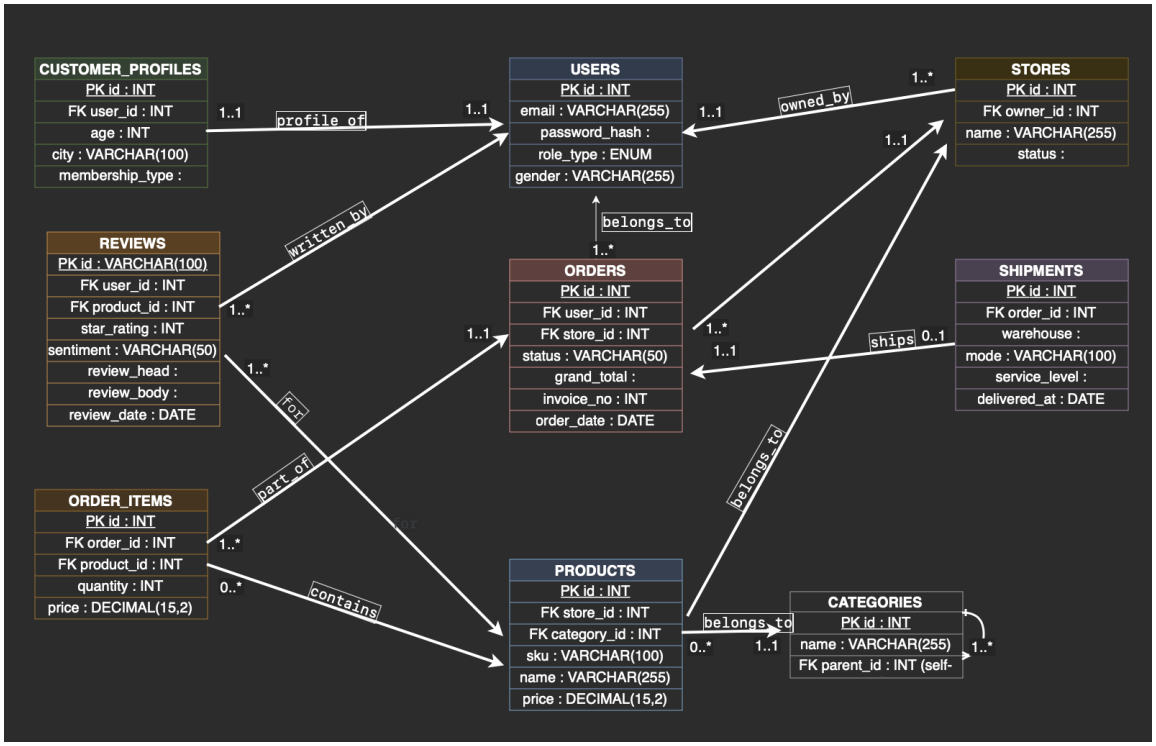

Figure 2 — Entity-Relationship Diagram (9 tables, 3NF)

3.3 Table Descriptions

The nine tables and their relationships are summarized below:

Table	Primary Key	Key Relationships & Notes
users	id (INT, AI)	Central entity; role_type ENUM (ADMIN/CORPORATE/INDIVIDUAL); UNIQUE idx_user_email
stores	id (INT, AI)	FK owner_id → users.id; status ENUM (ACTIVE/PASSIVE); owned by corporate users
products	id (INT, AI)	FK store_id → stores.id; FK category_id → categories.id; price DECIMAL(15,2)
orders	id (INT, AI)	FK user_id → users.id; FK store_id → stores.id; grand_total DECIMAL(20,2); invoice_no preserved
order_items	id (INT, AI)	FK order_id → orders.id; FK product_id → products.id; quantity + unit price
shipments	id (INT, AI)	FK order_id → orders.id; warehouse, mode (Flight/Ship/Road), delivered_at DATE
reviews	id (VARCHAR 100)	Amazon original review_id preserved; star_rating 1-5; sentiment; FK user_id, product_id
categories	id (INT, AI)	Self-referencing FK parent_id → categories.id; adjacency-list tree (4 roots, 14 sub)
customer_profiles	id (INT, AI)	FK user_id → users.id CASCADE DELETE; age, city, membership_type (Diamond–Bronze)

3.4 PostgreSQL Compatibility

The platform was designed to be portable to PostgreSQL. Key adaptations include: ENUM types replaced by VARCHAR with application-level validation; DECIMAL precision maintained at (15,2) and (20,2); JDBC URL updated to PostgreSQL driver with useUnicode=true&characterEncoding=UTF-8; InnoDB-specific directives removed from DDL. A single change in application.properties switches the target database.

4. ETL Process and Dataset Integration

4.1 Overview

Six heterogeneous Kaggle datasets were cleaned, normalized, and merged into the unified relational schema. Each dataset contributed to one or more target tables through a structured Extract-Transform-Load pipeline. The primary challenge was reconciling different column naming conventions, encodings, date formats, and identifier schemes across sources.

4.2 Data Integration Matrix

Target Table	DS1 Online Retail	DS2 Customer Behavior	DS3 Shipping	DS4 Amazon Sales	DS5 Pakistan E-Comm	DS6 Reviews

users	CustomerID	Demographics	—	—	CustomerID	—
products	StockCode, Desc	—	CostOfProduct	SKU, Style	SKU	ProductTitle
orders	Invoice No, Date	—	—	Status	Full Order	—
order_items	Quantity, UnitPrice	—	—	—	QtyOrdered	—
shipments	—	—	All fields	ShipServiceLevel	—	—
reviews	—	AvgRating	—	—	—	StarRating, Body
customer_profiles	—	Age, City, Membership	—	—	—	—
categories	—	—	—	Category	Category Name	ProductCategory

4.3 Data Cleaning Steps

- Missing Values: Rows dropped when required fields (email, sku, order_date) were absent; optional fields written as NULL
- Duplicate Removal: DISTINCT applied on email and sku to eliminate redundant records
- Date Standardization: All dates normalized to ISO 8601 (YYYY-MM-DD) format
- Currency Normalization: All prices converted to USD with DECIMAL(15,2) precision
- Encoding: utf8mb4_general_ci applied to all tables for Turkish and international character support
- Surrogate Key Generation: AUTO_INCREMENT surrogate keys generated for all tables; original identifiers (invoice_no, reviews.id) preserved for traceability
- Reference Resolution: Customer/order/product IDs across sources mapped to new surrogate keys to ensure FK integrity

5. User Interface Screens

The Angular frontend provides role-specific interfaces for each user type. Below are the key screens of the application.

5.1 Homepage & Product Store

The public-facing homepage features a hero banner with featured products, a product search bar, and best-seller highlights. Users can add items to their cart directly from the landing page before or after authentication.

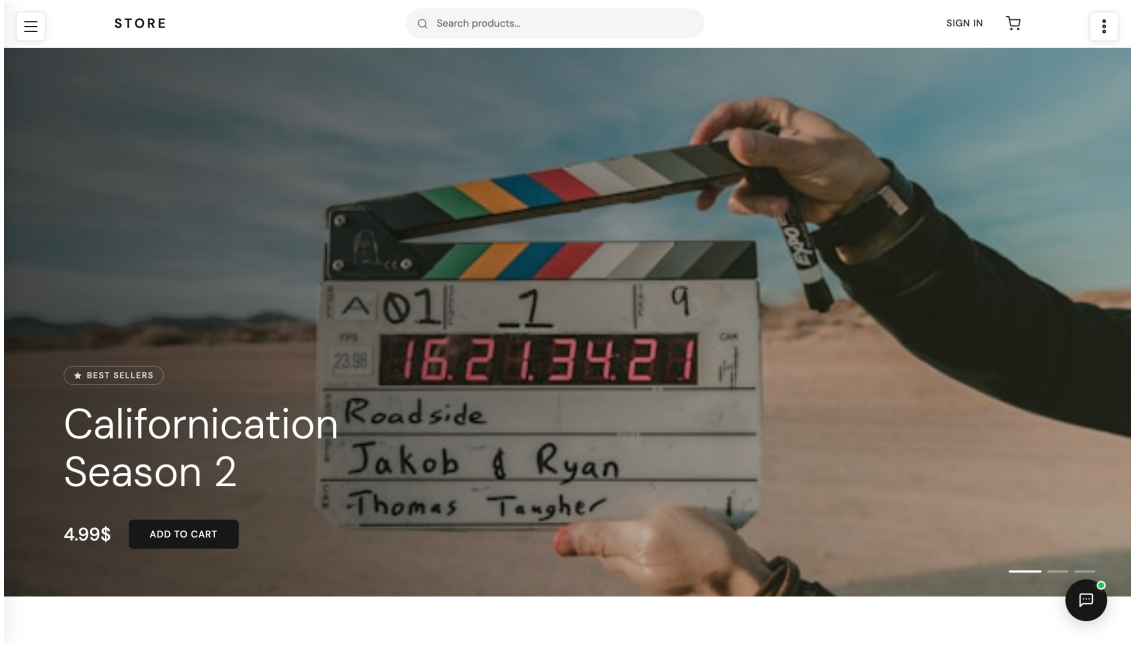

Figure 3 — Homepage with hero banner and featured product (Californication Season 2)

5.2 Login Page

The authentication screen presents a clean card-based layout with email and password fields. On successful login the backend returns a JWT token which is stored in the Angular HTTP interceptor for subsequent authenticated requests.

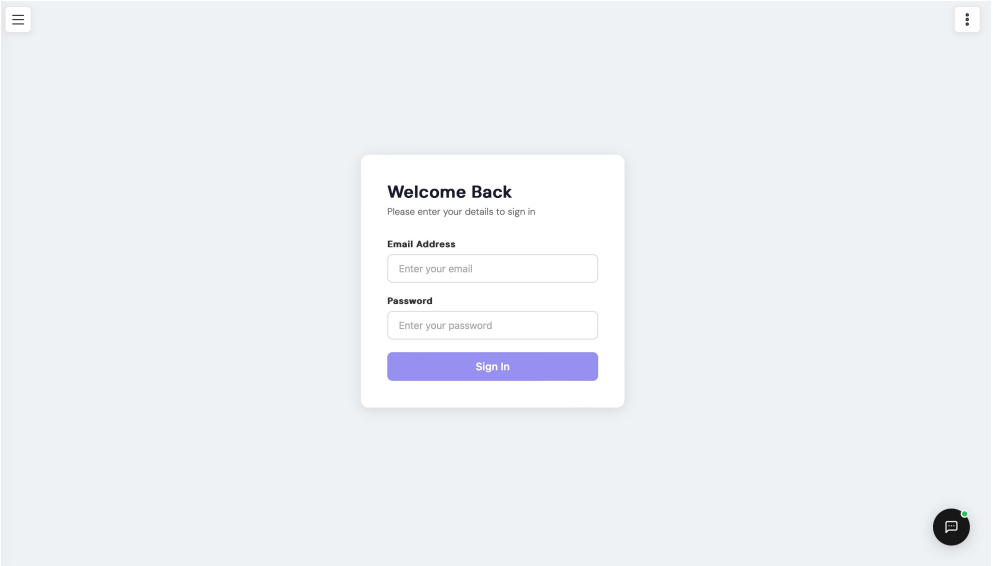

Figure 4 — Sign-in screen with JWT-based authentication

5.3 Admin Dashboard

The Admin dashboard provides a platform-wide overview showing total users, orders, revenue, and products. A doughnut chart visualises the relative distribution across these metrics. The admin can navigate to Manage Products and Manage Stores tabs and has access to the full-scope AI chatbot.

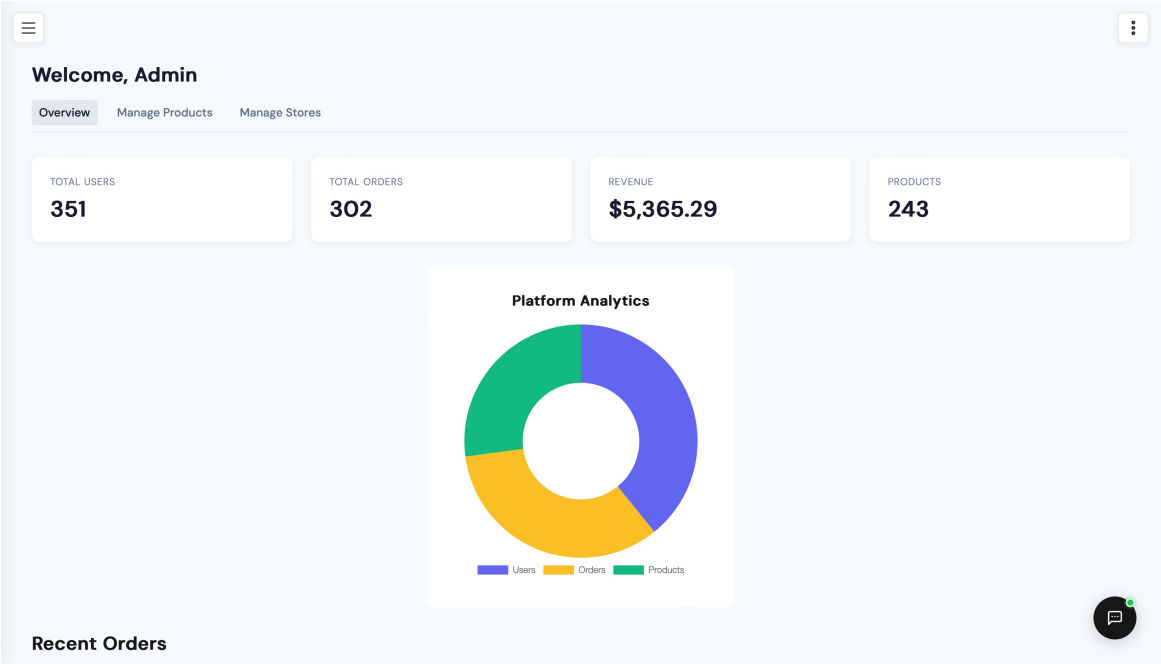

Figure 5 — Admin overview dashboard with platform analytics chart

5.4 Seller (Corporate) Dashboard

Corporate users see a seller-scoped overview limited to their own store data: product count, order count, revenue, and review count. An Order Status pie chart breaks down pending, shipped, and delivered orders, enabling sellers to monitor fulfilment performance at a glance.

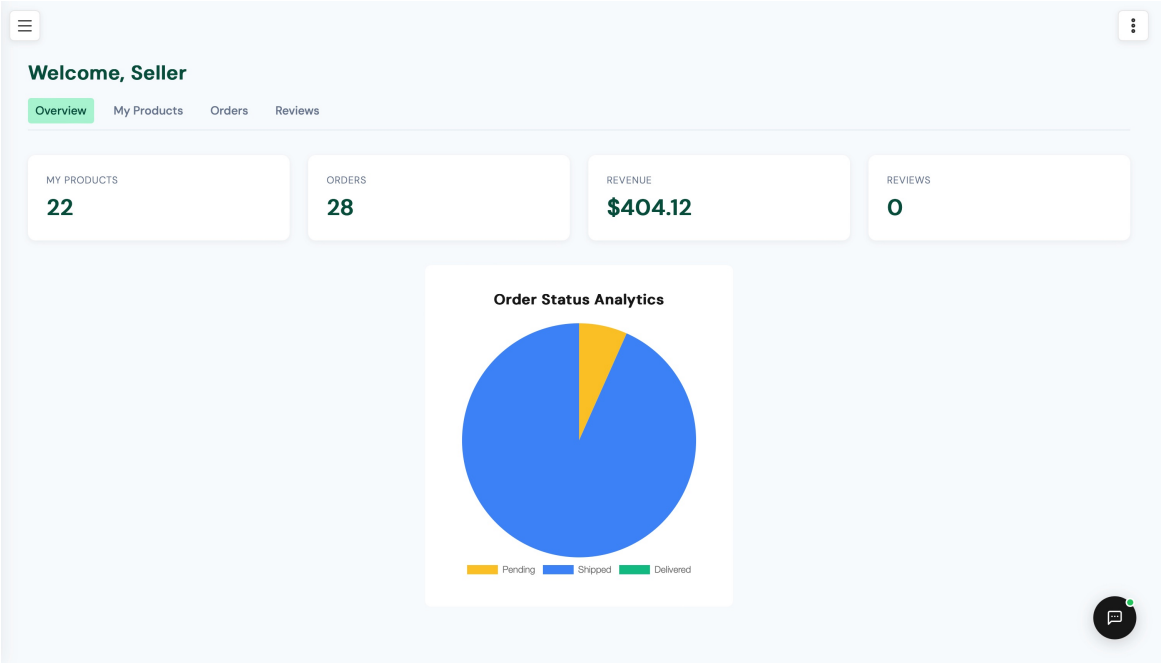

Figure 6 — Seller dashboard with order-status analytics

5.5 AI Assistant Chatbot

A floating chat widget is available on every page. For admin and corporate users the chatbot surfaces pre-defined quick-action chips (e.g. 'En cok harcama yapan 3 kisiyi getir') and accepts free-form natural language queries in both Turkish and English. Each message triggers the Text2SQL pipeline described in Section 6.

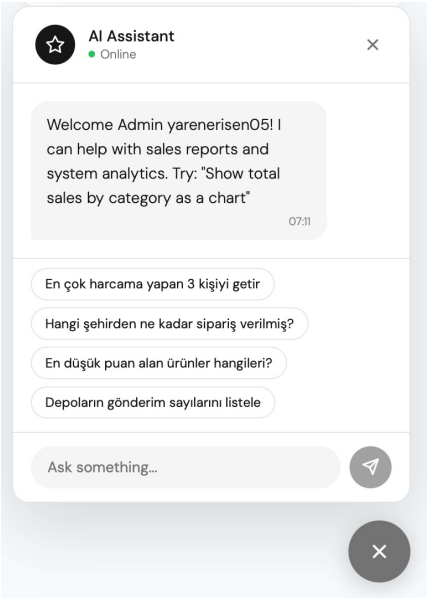

Figure 7 — AI Assistant chatbot with suggested query chips

5.6 Payment (Stripe Checkout)

The checkout screen integrates Stripe.js to render a secure card-input element directly within the Angular SPA. The total amount is displayed in blue, the card number field is hosted by Stripe, and a 'Pay Now' button submits the payment intent. In production the test mode banner is removed and live Stripe keys are injected via environment configuration.

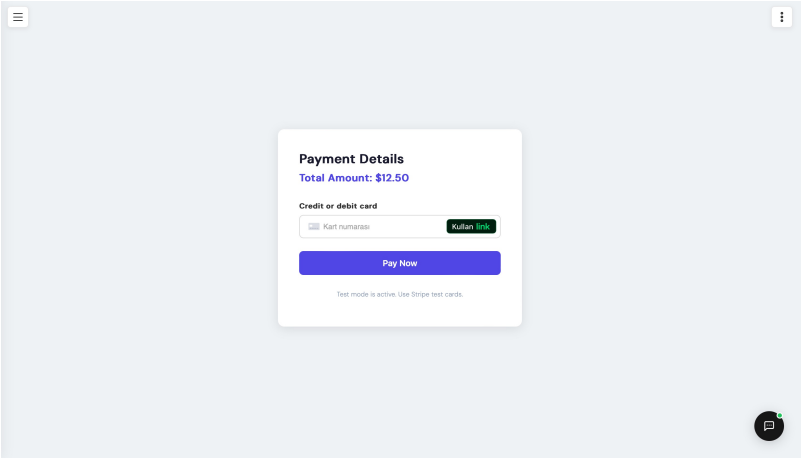

Figure 8 — Stripe-powered payment details screen

6. API Documentation

6.1 Endpoint Inventory

All endpoints return JSON and are protected by JWT bearer tokens except the login and public product endpoints. The Angular client communicates via CORS-enabled requests from http://localhost:4200.

Controller	HTTP Path	Method(s)	Access
AuthController	/api/auth/login	POST	Public — returns JWT on valid credentials
UserController	/api/users	GET/POST/PUT/DELETE	ADMIN only
StoreController	/api/stores	GET/POST/PUT/DELETE	ADMIN, CORPORATE
ProductController	/products	GET	Public read; ?bestseller=true returns top 3
CategoryController	/api/categories	GET/POST	Mixed
OrderController	/api/orders	GET	Authenticated
OrderItemController	/api/orderitems	GET	Authenticated
ShipmentController	/api/shipments	GET	Mixed
ReviewController	/api/reviews	GET/POST	INDIVIDUAL can POST
CustomerProfileController	/api/customerprofiles	GET	Authenticated
ChatController	/api/chat/ask	GET	All roles — Text2SQL pipeline

6.2 Authentication Flow

The client POSTs credentials to /api/auth/login. AuthController calls UserRepository.findByEmail(), verifies the password, and delegates to JwtUtils.generateToken(email). The HS256-signed token (24-hour validity, secret from app.jwt.secret) is returned in the response body. Subsequent requests pass the token in the Authorization: Bearer <token> header; JwtAuthFilter intercepts every request, validates the token, and populates the Spring SecurityContext.

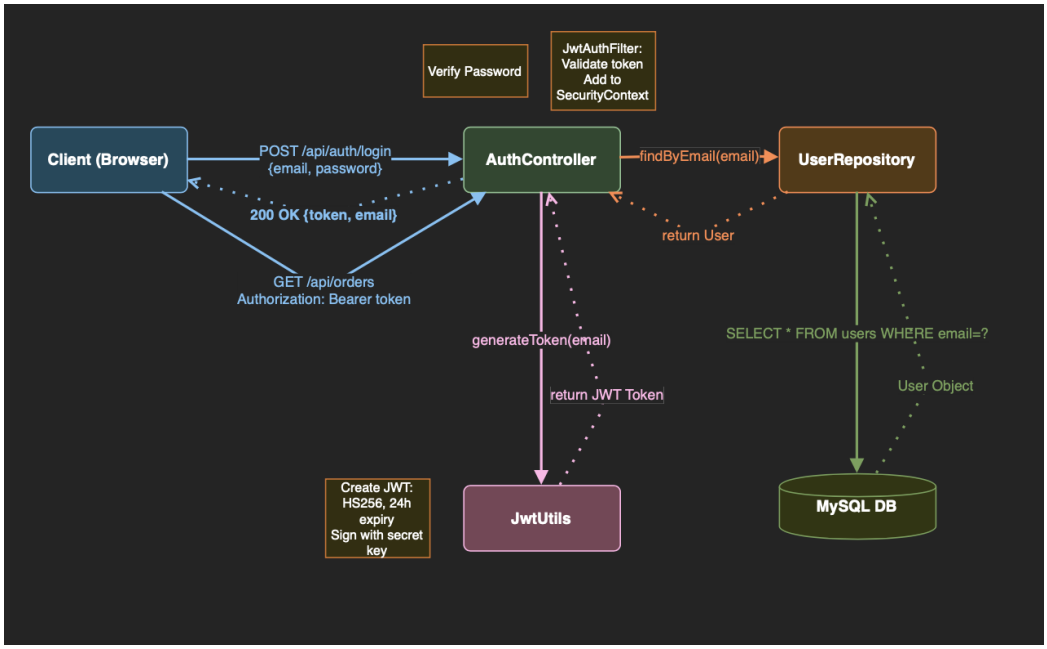

Figure 9 — JWT Authentication Flow Diagram

6.3 Error Handling

HTTP Status	Scenario
401 Unauthorized	Invalid password or missing/expired JWT token
404 Not Found	User, product, or resource not found (RuntimeException)
500 Internal Server Error	Unhandled server-side exception
AI Blocked (200)	Sensitive keyword detected — returns human-readable security message

7. AI Chatbot Architecture (Text2SQL)

7.1 Problem Definition

Sales and marketing teams cannot be expected to write SQL, yet they require fast, data-driven insights. The AI Assistant converts natural language questions into MySQL SELECT statements, executes them securely via JdbcTemplate, and returns results as a readable Markdown analysis report.

7.2 Multi-Agent Pipeline

ChatService implements a logically multi-agent pipeline where each stage has a single, well-defined responsibility:

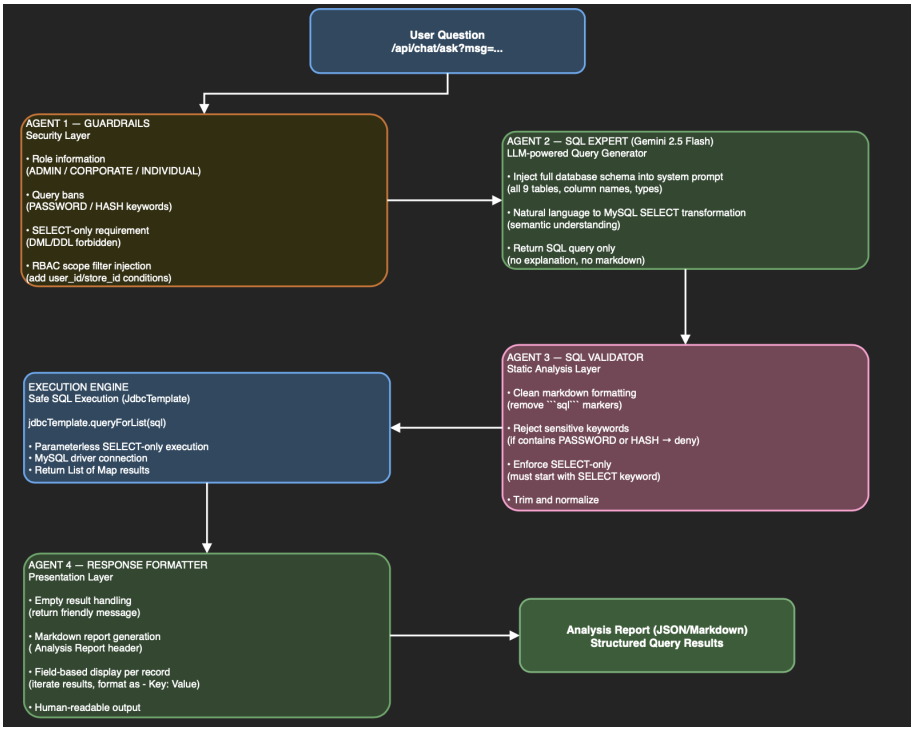

Figure 10 — Multi-Agent Text2SQL Architecture

Agent	Role	Responsibility
Guardrails Agent	Security & Scope Manager	Enriches the question with role-based restrictions; injects RBAC scope filters (user_id / store_id conditions)
SQL Expert Agent	Gemini 2.5 Flash	Receives the full 9-table schema in the system prompt; generates a schema-aware MySQL SELECT query
SQL Validator	Static Analyser	Strips markdown fences; rejects queries containing PASSWORD or HASH; enforces SELECT-only; trims output
Execution Engine	JdbcTemplate	Executes parameterless SELECT against MySQL; returns List<Map<String,Object>> result rows
Response Formatter	Data Analyst	Converts raw rows into a human-friendly Markdown Analysis Report; handles empty results gracefully

7.3 Role-Based Data Access

Role	Access Scope	Implementation Logic
Individual	Own orders, purchases, reviews, and spending patterns only	Appends WHERE user_id = (SELECT id FROM users WHERE email = '...')
Corporate	Products, orders, customers, and reviews of their own store	Appends WHERE store_id IN (SELECT id FROM stores WHERE owner_id = ...)
Admin	Full platform — all tables, all rows	No filter injected

7.4 Gemini API Integration

ChatService sends the full database schema as a system prompt to the Gemini 2.5 Flash endpoint on every request. The prompt injects the user's email and role, instructs the model to return only valid MySQL SELECT queries, and explicitly forbids returning password or hash data. The generated SQL is cleaned, validated, and executed; results are then formatted by the Response Formatter agent and returned as a Markdown string to the Angular frontend.

7.5 Example Interactions

Natural Language Question	Pipeline Outcome
Top 5 best-selling products	SQL with ORDER BY SUM(oi.quantity) DESC LIMIT 5; Markdown table with product names and totals
Category-based sales last month	GROUP BY c.name with date filter; bar-chart data returned
Lowest-rated products	ORDER BY AVG(r.star_rating) ASC; table of product names and ratings
How many orders shipped by air?	COUNT(*) WHERE shipments.mode = 'Flight'
Hello!	Guardrails agent intercepts; returns greeting without SQL generation
List all password hashes	Validator blocks; returns security-protocol message

8. Data Visualization

8.1 Chart.js — Dashboard Analytics

Chart.js is embedded in the Angular dashboard components to render lightweight, animated charts. The Admin dashboard uses a doughnut chart to compare platform-wide totals (users, orders, products). The Seller dashboard renders a pie chart showing order-status distribution (Pending / Shipped / Delivered). Data is fetched from the REST API on component initialization and bound to Chart.js datasets via Angular lifecycle hooks.

- Chart types used: Doughnut, Pie (dashboard overviews)
- Data binding: RxJS Observable → Angular component → Chart.js datasets
- Color scheme: Blue (#6366F1 / #4ECDC4), Amber (#FFB347), Green (#22C55E)

8.2 Plotly — AI-Driven Analytics

Plotly is integrated for richer interactive analytics triggered by the AI chatbot. When an admin or corporate user sends a query requesting chart output (e.g. 'Show total sales by category as a chart'), the Response Formatter detects chart-intent and returns a Plotly-compatible JSON configuration. The Angular ChatComponent renders this configuration as an interactive bar or line chart inline within the conversation.

- Chart types used: Bar (category sales, revenue over time), Scatter (customer segments)
- Interactivity: Hover tooltips, zoom, pan, and export to PNG
- Integration path: Gemini response → JSON chart config → Angular Plotly component

8.3 Visualization Architecture

Library	Use Case	Data Source
Chart.js	Admin & Seller static dashboard KPIs	REST API endpoints called on page load
Plotly	AI chatbot dynamic analytics	Gemini response formatter with chart-intent detection

9. Payment Integration — Stripe API

9.1 Overview

Checkout is handled by the Stripe API, providing PCI-compliant card collection without sensitive data touching the application server. Stripe.js loads the Elements library which renders a hosted card-number input field directly in the Angular SPA. The server creates a Stripe PaymentIntent and returns the client_secret, which the frontend uses to confirm the payment.

9.2 Payment Flow

Step	Description
1 — Cart Review	User reviews cart total; clicks 'Proceed to Checkout'
2 — PaymentIntent Creation	Angular calls backend POST /api/payments/create-intent; Spring creates Stripe PaymentIntent with amount and currency
3 — Stripe Elements Render	Frontend loads Stripe.js; mounts card Element into Angular component; test mode active
4 — Payment Confirmation	User enters card details (Stripe-hosted iframe); Angular calls stripe.confirmCardPayment(clientSecret)
5 — Webhook / Status Update	Stripe webhook notifies backend on success; order status updated to 'Paid'
6 — Live Migration	Replace test keys with live Stripe keys in environment.prod.ts; remove test-mode banner

9.3 Configuration

Stripe keys are stored in Angular environment files (environment.ts / environment.prod.ts) and in Spring application.properties (stripe.secret.key). The secret key is excluded from Git via .gitignore. Test card 4242 4242 4242 4242 can be used with any future expiry and CVC to verify the integration.

10. Security Architecture

10.1 Layered Defense

```
// ChatService.java (summary)
String URL = "https://generativelanguage.googleapis.com/v1beta/" +
             "models/gemini-2.5-flash:generateContent?key=" + API_KEY;

Map<String, Object> request = Map.of(
    "contents", List.of(
        Map.of("parts", List.of(
            Map.of("text", systemPrompt)
        ))
    )
);

Map<String, Object> response = restTemplate.postForObject(URL, request, Map.class);
String generatedSql = ((Map)((Map)((List)response.get("candidates")).get(0))
                        .get("content")).get("parts")...;

// Clean markdown fences
generatedSql = generatedSql.replace("```sql", "").replace("```", "").trim();

// Validator: forbidden keyword + SELECT-only
if (upperSql.contains("PASSWORD") || upperSql.contains("HASH")
    || !upperSql.startsWith("SELECT")) {
    return "⚠️ Access to this data was blocked due to security protocol.";
}

List<Map<String, Object>> result = jdbcTemplate.queryForList(generatedSql);
return formatResult(result);
```

Figure 11 — Multi-Agent Chatbot Security Layer Diagram

Layer	Security Measure
Transport	CORS restricted to http://localhost:4200; production: HTTPS + strict origin policy
Authentication	JWT (HS256); JwtAuthFilter validates every incoming request and populates SecurityContext
Authorization	Spring Security SecurityFilterChain; endpoint-level role enforcement
AI Guardrails	System-prompt rules + post-generation static keyword validator
Database (CRUD)	JPA prepared statements; no raw string concatenation; method-based queries
Database (AI)	JdbcTemplate SELECT-only executor; DML and DDL keywords blocked
Sensitive Data	password_hash blocked in all AI responses; application.properties excluded from Git
Stripe	PCI-compliant Stripe Elements; card data never touches application server

10.2 Attack Vectors and Countermeasures

Attack Vector	Risk	Countermeasure
SQL Injection (CRUD)	Data exfiltration / modification via REST endpoints	JPA method-based queries enforce parameter binding; no raw concatenation
SQL Injection (AI)	Malicious SQL via crafted natural language	SELECT-only executor; DML/DDL keywords blocked post-generation
Prompt Injection	Overriding system prompt via user input	Security rules re-injected on every request; user input appended at prompt end
Sensitive Data Leakage	Exposure of password_hash or PII via chatbot	Post-generation validator rejects any output containing PASSWORD or HASH
JWT Theft	Replay attacks with stolen tokens	24-hour validity; HTTPS enforced in production; refresh token infrastructure planned
API Key Exposure	Gemini / Stripe key leaked to version control	Keys stored in application.properties excluded from Git via .gitignore

10.3 Known Limitations

In the current development build, SecurityConfig applies permitAll to all endpoints for convenience. For production deployment the following hardening steps are required:

- Enable role-based requestMatchers rules: .hasRole('ADMIN') on admin endpoints
- Migrate password storage to BCryptPasswordEncoder
- Obtain user email and role from the Authentication object in ChatController (not from request parameter)
- Enable HTTPS with a valid TLS certificate and enforce strict CORS origins

11. Challenges Encountered and Solutions

Challenge	Problem Description	Solution Applied
Heterogeneous Dataset Integration	Six Kaggle datasets with different column names,	Designed target schema first; created field mapping matrix; ISO 8601 dates; USD currency; utf8mb4 encoding; surrogate key generation

	encodings, date formats, and ID structures	
Turkish Character Corruption	City names such as Istanbul and Umraniye rendered as '?' in early ETL runs	DEFAULT CHARSET=utf8mb4; useUnicode=true&characterEncoding=UTF-8 in JDBC URL; ChatController returns text/plain;charset=UTF-8
LLM Hallucination in SQL	Gemini generated non-existent table and column names when schema was not provided	Full 9-table schema injected into system prompt on every ChatService request
LLM Output Cleaning	Generated SQL wrapped in sql ... markdown fences; JdbcTemplate threw syntax errors	String-level removal of sql and fence patterns; trim(); static validator enforces SELECT-start
Prompt Injection	Users could craft natural language to extract password_hash data	Two-layer defence: system prompt rule + post-generation static keyword check (PASSWORD / HASH)
API Key in Git History	Gemini API key entered version control history during development	application.properties added to .gitignore (commit dc55a54); key removed from tracking; README instructions added
Gemini Response Parsing	Nested JSON structure candidates[0].content.parts[0].text caused ClassCastException	RestTemplate reads Map<String,Object>; safe (List) and (Map) casts applied; single try/catch for user-friendly error
Complex JOIN Performance	AI-generated queries joining 8+ tables exceeded acceptable response times	Index on users.email; InnoDB auto-generates FK indexes; Spring Data JPA @Index on User entity
Multi-Tenancy RBAC	Corporate users could potentially access data from other stores	store_id IN (SELECT id FROM stores WHERE owner_id=...) filter injected both in REST security config and chatbot prompt

12. Conclusion and Future Improvements

12.1 Summary

This project demonstrates how a modern e-commerce business intelligence platform can be built end-to-end by a small team: normalizing heterogeneous data sources through a structured ETL pipeline, securing access with JWT-based RBAC, building a responsive Angular SPA, integrating Stripe for payments, delivering interactive analytics through Chart.js and Plotly, and powering natural language data access with a multi-agent Text2SQL chatbot built on Google Gemini 2.5 Flash. The Spring Boot + Angular + Gemini combination validates the 'large capability with a small team' principle for modern full-stack development.

12.2 Achieved Goals

- 9-table relational schema normalized to 3NF, MySQL-primary with PostgreSQL compatibility
- ETL integration of six Kaggle datasets with full field mapping and data cleaning
- JWT-based stateless authentication with 11 REST controllers
- Role-specific dashboards for Admin, Corporate (Seller), and Individual users
- Natural language query → SQL → formatted Markdown response pipeline (Text2SQL)
- Stripe API payment integration with test mode and live-ready configuration
- Chart.js dashboard charts and Plotly AI-driven interactive analytics
- Multi-layer security (RBAC + prompt guardrails + static SQL validator)